Architecture, Distributed Systems & Data Engineering

Senior / Lead Engineer Interview Study Guide

---

0. Objetivo del documento

El objetivo de este documento es desarrollar la capacidad para:

- Diseñar arquitecturas de aplicaciones.
- Diseñar sistemas distribuidos.
- Diseñar arquitecturas orientadas a eventos.
- Entender profundamente Kafka y sistemas de mensajería.
- Diseñar pipelines de datos batch y streaming.
- Entender Data Lake, Data Warehouse y Lakehouse.
- Diseñar pipelines utilizando CDC.
- Seleccionar bases de datos según el problema.
- Aplicar patrones de resiliencia y consistencia.
- Diseñar soluciones escalables, observables y tolerantes a fallos.
- Explicar decisiones arquitectónicas y sus trade-offs.
- Resolver ejercicios de system design mediante diagramas.

La meta no es memorizar definiciones.

La meta es poder contestar:

> **What problem are we solving?**
>
> **What architecture would you choose?**
>
> **Why?**
>
> **What are the trade-offs?**
>
> **What happens when something fails?**
>
> **How does the system scale?**
>
> **How do we guarantee consistency?**
>
> **How do we monitor it?**

1. SOFTWARE & SOLUTION ARCHITECTURE

1.1 Software Architecture

Software Architecture define la estructura interna de una aplicación y las relaciones entre sus componentes.

Una arquitectura debe responder:

- ¿Qué componentes existen?
- ¿Qué responsabilidad tiene cada componente?
- ¿Cómo se comunican?
- ¿Dónde viven los datos?
- ¿Cómo escalan?
- ¿Qué ocurre cuando un componente falla?

Ejemplo:

```
None
                Client
                   |
                   v
               API Gateway
                   |
          +--------+--------+
          |                 |
          v                 v
   Payment Service    Order Service
          |                 |
          v                 v
     Payment DB        Order DB
```

La arquitectura no solamente describe componentes. También describe:

- responsabilidades;
- dependencias;
- flujos de información;
- límites;
- restricciones;

- comportamiento ante fallos.

1.2 Solution Architecture

Solution Architecture conecta los requerimientos de negocio con una solución tecnológica.

El arquitecto debe identificar:

Functional Requirements

¿Qué debe hacer el sistema?

Ejemplo:

- El usuario puede realizar un pago.
- El sistema debe registrar la transacción.
- El usuario debe recibir confirmación.

Non-Functional Requirements

¿Cómo debe comportarse?

Ejemplo:

- 10,000 TPS.
- Latencia menor a 200 ms.
- 99.99% availability.
- Recovery Time Objective de 30 minutos.
- Datos auditables durante 7 años.

1.3 System Architecture

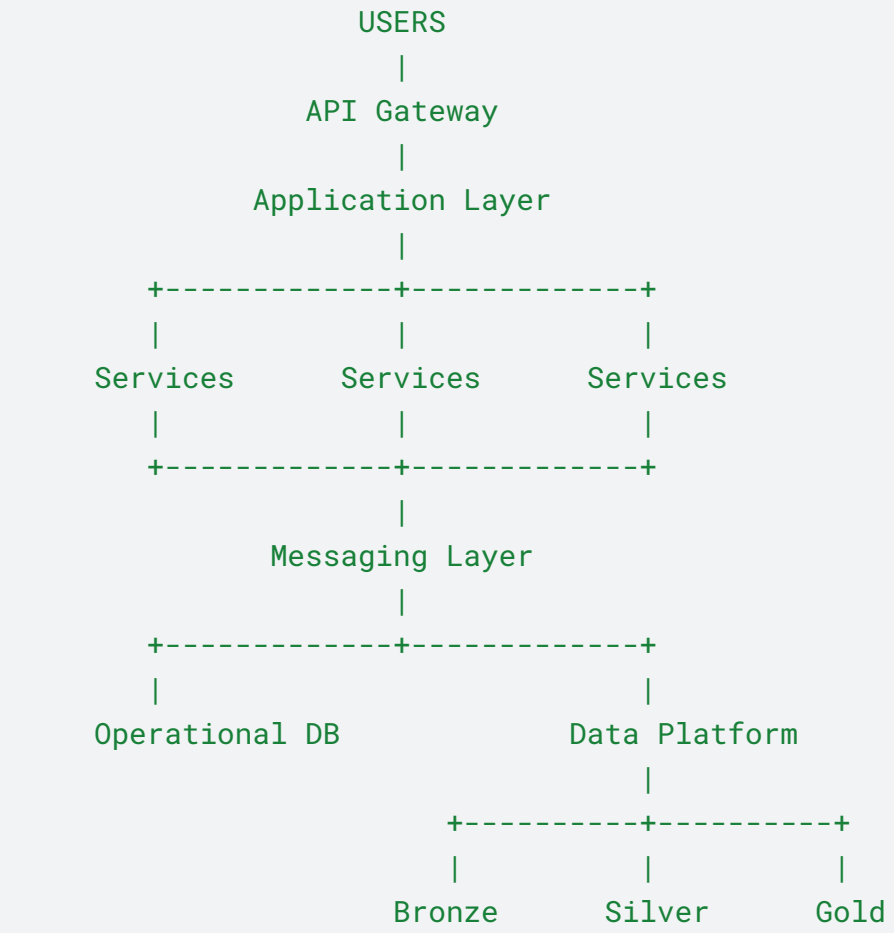
System Architecture contempla el sistema completo:

```
None
Application
Infrastructure
Network
Databases
Messaging
Storage
```

```
Security
Data Pipelines
Monitoring
Operations
```

Una arquitectura empresarial puede verse como:

None



---

2. ARCHITECTURE STYLES

2.1 Monolithic Architecture

Toda la aplicación se despliega como una unidad.

Ventajas:

- Simple de desarrollar inicialmente.
- Simple de desplegar.
- Fácil comunicación entre componentes.
- Menor complejidad operacional.

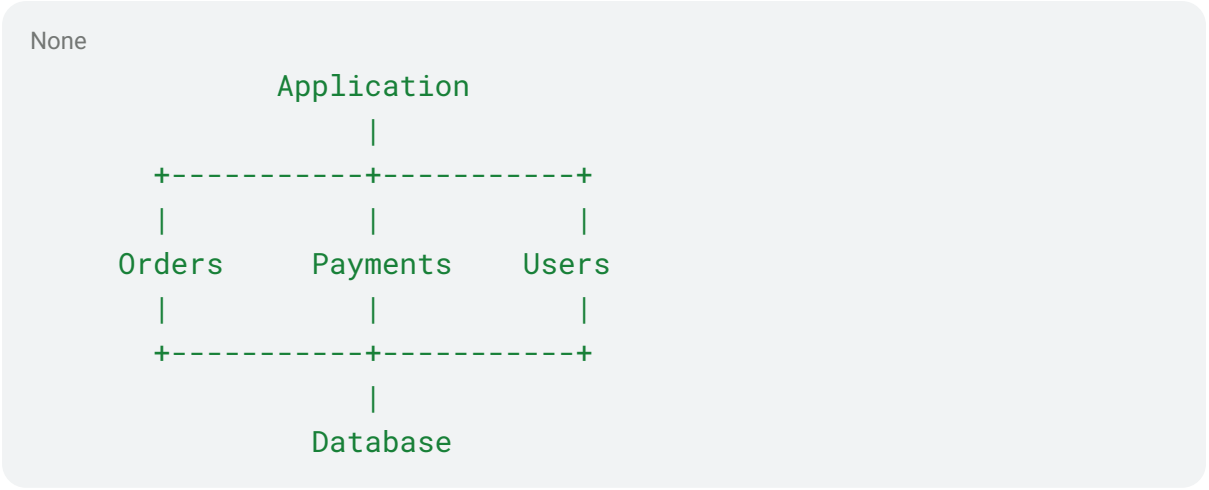
Desventajas:

- Escalamiento menos granular.
- Deploy de una pequeña funcionalidad puede requerir desplegar toda la aplicación.
- Mayor acoplamiento cuando crece.
- Un problema puede afectar múltiples funcionalidades.

2.2 Modular Monolith

Una sola aplicación desplegable, pero internamente dividida en módulos bien definidos.

None



Es importante entender:

> Modular Monolith ≠ Microservices.

El modular monolith mantiene una unidad de deployment.

2.3 Microservices

Una aplicación se divide en servicios independientes.

```
None
```

```
             API Gateway
                  |
       +----------+----------+
       |          |          |
   Orders     Payments    Users
       |          |          |
   DB Orders  DB Payments DB Users
```

Características:

- Independently deployable.
- Independently scalable.
- Business/domain boundaries.
- Service ownership.
- Communication through APIs/events.

Pero introducen problemas:

- Network failures.
- Distributed transactions.
- Distributed tracing.
- Service discovery.
- Eventual consistency.
- Deployment complexity.
- Operational overhead.

Una pregunta frecuente:

> Why not use microservices everywhere?

Porque microservices no eliminan complejidad; **mueven la complejidad del código hacia la red y la operación distribuida**.

2.4 Event-Driven Architecture

Los componentes se comunican mediante eventos.

```
None
```

```
Order Service
      |
```

```
      | OrderCreated
      v
    Kafka
      |
 +----+---------+
 |              |
 v              v
Payment      Inventory
Service      Service
```

Ventajas:

- Loose coupling.
- Asynchronous processing.
- Scalability.
- Replayability cuando el broker lo soporta.
- Múltiples consumidores.

Desventajas:

- Eventual consistency.
- Debugging más complejo.
- Ordering challenges.
- Duplicate events.
- Schema evolution.
- Operational complexity.

2.5 Layered Architecture

```
None
Presentation
      |
Business Logic
      |
Data Access
      |
Database
```

Es simple y común, pero hay que controlar que las capas no se acoplen incorrectamente.

2.6 Hexagonal Architecture

También conocida como Ports and Adapters.

El dominio queda aislado de infraestructura.

```
None
              REST Adapter
                   |
                   v
         +--------------------+
         |                    |
DB ---> |   Domain/Core      | <--- External API
         |                    |
         +--------------------+
                   |
              Message Adapter
```

El dominio conoce interfaces/ports, no implementaciones concretas.

2.7 Clean Architecture

Principio fundamental:

> Dependencies should point inward.

El business logic no debe depender directamente de:

- databases;
- frameworks;
- UI;
- external APIs.

Esto facilita testing y reemplazo de infraestructura.

---

3. ARCHITECTURE QUALITY ATTRIBUTES

Las decisiones arquitectónicas se basan principalmente en requisitos funcionales y no funcionales.

Scalability

Capacidad de soportar crecimiento.

Vertical Scaling

Más CPU/RAM.

Horizontal Scaling

Más instancias.

```
None
          Load Balancer
          /     |      \
        App    App     App
```

Performance

Qué tan rápido responde el sistema.

Métricas:

- Latency.
- Throughput.
- CPU.
- Memory.
- I/O.

Availability

Probabilidad de que el sistema esté disponible.

Ejemplos:

- 99%
- 99.9%
- 99.99%
- 99.999%

Mientras más "nines", normalmente mayor costo y complejidad.

Reliability

Probabilidad de que el sistema ejecute correctamente su función durante un período determinado.

Availability y Reliability no son exactamente lo mismo.

Un sistema puede estar disponible pero producir resultados incorrectos.

Resilience

Capacidad de absorber fallos y recuperarse.

Ejemplo:

```
None
Service A
   |
   v
Service B ---- DOWN
   |
   +----> Timeout
   |
   +----> Circuit Breaker
   |
   +----> Fallback
```

Fault Tolerance

Capacidad de continuar operando ante ciertos fallos sin interrupción significativa.

Observability

Capacidad de inferir el estado interno de un sistema a partir de sus outputs.

Tres pilares:

- Logs.
- Metrics.
- Traces.

Maintainability

Facilidad para modificar y mantener el sistema.

Factores:

- Low coupling.
- High cohesion.
- Modularity.
- Automated testing.
- Documentation.

---

4. DISTRIBUTED SYSTEMS

Esta es una de las áreas más importantes para la entrevista.

Un distributed system está compuesto por múltiples procesos/nodos que colaboran mediante una red.

El problema fundamental:

> **La red puede fallar.**

4.1 Partial Failure

En un sistema local:

```
None
Application -> Database
```

Si la aplicación está viva y la DB está viva, normalmente funciona.

En un sistema distribuido:

```
None
Service A
    |
    X
Network
    |
Service B
```

Service B puede estar funcionando, pero:

- la red puede estar caída;
- el request puede perderse;
- la respuesta puede perderse;
- puede existir timeout;
- el request pudo ejecutarse aunque no recibimos respuesta.

Esto genera problemas de:

- retries;
- duplicates;
- idempotency;
- consistency.

4.2 CAP Theorem

CAP habla de:

- Consistency.
- Availability.
- Partition tolerance.

Cuando ocurre una network partition, un sistema distribuido debe hacer un trade-off entre consistency y availability.

La idea importante para entrevista:

> Partition tolerance is not really optional in a distributed system.

La pregunta relevante es:

> During a partition, do we prefer consistency or availability?

4.3 Strong vs Eventual Consistency

Strong Consistency

Después de una escritura, los lectores observan inmediatamente el valor actualizado.

Eventual Consistency

Los nodos pueden temporalmente observar diferentes estados, pero eventualmente convergen.

```
None
Time 0
Node A = 10
Node B = 10

Write A = 20

Time 1
Node A = 20
Node B = 10

Time 2
Node A = 20
Node B = 20
```

Eventual consistency aparece frecuentemente en arquitecturas:

- distributed databases;
- caches;
- event-driven systems;
- replicated systems.

---

5. MICROSERVICES

5.1 Service Boundaries

El objetivo no es simplemente dividir una aplicación en muchos servicios.

Hay que encontrar boundaries adecuados.

Un servicio debería representar una capacidad de negocio coherente.

Ejemplo:

```
None
Orders
Payments
Inventory
Shipping
Customer
```

5.2 Database per Service

Cada servicio debería controlar su propio modelo de persistencia cuando se busca verdadero aislamiento.

```
None
Order Service ---> Order DB

Payment Service -> Payment DB

Inventory ------> Inventory DB
```

Esto evita que diferentes servicios dependan directamente de las mismas tablas.

Pero introduce:

- Distributed transactions.

5.3 Service Discovery

Permite localizar dinámicamente servicios.

Ejemplo:

```
None
Payment Service
      |
      v
Service Discovery
      |
      v
Payment Instance A
Payment Instance B
Payment Instance C
```

Puede implementarse mediante:

- Kubernetes service discovery.
- DNS.
- Service registry.
- Service mesh.

5.4 API Gateway

Centraliza capacidades como:

- routing;
- authentication;
- rate limiting;
- TLS termination;
- request transformation;
- observability.

---

6. RESILIENCE PATTERNS

Esta sección debe estudiarse muy bien.

Timeout

Nunca esperar indefinidamente por una dependencia.

```
None
Service A ---> Service B

       wait 2 sec

       TIMEOUT
```

Retry

Reintentar una operación cuando el error puede ser transitorio.

No todo debe reintentarse.

No es buena idea hacer retry indiscriminadamente sobre:

- non-idempotent operations;
- validation errors;
- authentication failures;
- permanent failures.

Exponential Backoff

```
None
Retry 1 -> 100 ms
Retry 2 -> 200 ms
Retry 3 -> 400 ms
Retry 4 -> 800 ms
```

Jitter

Agrega variabilidad al tiempo de retry.

Evita:

```
None
Instance A -> retry at 1s
Instance B -> retry at 1s
Instance C -> retry at 1s
Instance D -> retry at 1s
```

Todos golpean nuevamente el servicio al mismo tiempo.

Circuit Breaker

Estados:

```
None
CLOSED
   |
   | failures exceed threshold
   v
OPEN
   |
   | timeout
   v
HALF-OPEN
   |
   +---- success ---> CLOSED
   |
   +---- failure ---> OPEN
```

Objetivo:

> Prevent cascading failures.

Bulkhead

Aislar recursos.

```
None
Application

+------------------+
| Payment Threads  |
+------------------+

+------------------+
| Report Threads   |
+------------------+
```

Si reporting falla, no debería consumir todos los recursos necesarios para payments.

Rate Limiting

Controlar cantidad de requests.

Ejemplo:

```
None
100 requests / minute / client
```

Load Shedding

Cuando el sistema está saturado puede rechazar trabajo de baja prioridad para proteger el sistema completo.

---

7. DISTRIBUTED TRANSACTIONS

7.1 El problema

Supongamos:

```
None
Order Service
Payment Service
Inventory Service
```

Una operación requiere:

1. Crear Order.
2. Reservar Inventory.
3. Procesar Payment.

Cada servicio tiene su propia DB.

No podemos simplemente ejecutar:

```
None
BEGIN TRANSACTION
...
COMMIT
```

sobre tres bases independientes de manera sencilla.

7.2 Two Phase Commit

Dos fases:

Phase 1 — Prepare

```
None
Coordinator
    |
    +----> Service A: Ready?
    |
    +----> Service B: Ready?
```

Phase 2 — Commit

Si todos responden YES:

```
None
Coordinator
    |
    +----> Commit A
    |
    +----> Commit B
```

Problemas:

- Blocking.
- Coordinator failure.
- Locking.
- Latency.
- Scalability.

7.3 Saga Pattern

Saga divide la operación en transacciones locales.

```
None
Create Order
     |
     v
Reserve Inventory
     |
     v
Process Payment
```

Si falla Payment:

```
None
Cancel Order
     |
     v
Release Inventory
```

Estas acciones son:

> Compensating Transactions.

7.4 Saga Orchestration

Existe un coordinador.

```
None
              Saga Orchestrator
               /       |       \
              v        v        v
          Order    Inventory  Payment
```

Ventajas:

- Flujo explícito.
- Fácil de visualizar.
- Control central.
- Bueno para workflows complejos.

Trade-off:

- Orchestrator conoce los participantes.
- Puede convertirse en punto de complejidad.

7.5 Saga Choreography

No existe un coordinador central.

```
None
OrderCreated
     |
     v
Inventory
     |
StockReserved
     |
     v
Payment
```

Cada servicio reacciona a eventos.

Ventajas:

- Loose coupling.
- Autonomía.

Problemas:

- Flujo más difícil de entender.
- Debugging complejo.
- Puede crear event chains difíciles de mantener.

---

8. OUTBOX / INBOX / IDEMPOTENCY

Dual Write Problem

Problema:

```
None
DB Write
   |
   +------> Success

Kafka Publish
   |
   +------> Failure
```

Ahora la DB tiene el cambio pero Kafka no.

Transactional Outbox

```
None
Application
     |
     v
+-----------------------+
```

```
| DB Transaction        |
|                       |
| Business Table        |
| Outbox Table          |
+-----------------------+
           |
           v
      Outbox Relay
           |
           v
         Kafka
```

La actualización de negocio y el evento se escriben en la misma transacción local.

Inbox Pattern

El consumidor registra primero el mensaje recibido.

```
None
Kafka
  |
  v
Inbox Table
  |
  v
Processor
  |
  v
Business DB
```

Esto ayuda a manejar:

- retries;
- duplicates;
- crash recovery.

Idempotent Consumer

Un consumidor idempotente produce el mismo resultado aunque procese el mismo mensaje varias veces.

Ejemplo:

```
None
PaymentCompleted event
event_id = 123
```

Si llega dos veces:

```
None
123 -> process
123 -> ignore
```

Idempotency Key

Especialmente importante en payments.

```
None
Idempotency-Key: abc-123
```

Si el mismo request llega nuevamente:

```
None
abc-123 already processed
```

El sistema devuelve el resultado anterior en lugar de ejecutar nuevamente el payment.

---

9. EVENT-DRIVEN ARCHITECTURE

Event vs Command

Event

Describe algo que ya ocurrió.

```
None
PaymentCompleted
OrderCreated
InventoryReserved
```

Command

Solicita que ocurra una acción.

```
None
ProcessPayment
ReserveInventory
CancelOrder
```

Una buena pregunta de entrevista:

> "Is this message describing something that happened or requesting something to happen?"

9.1 Event Contract

Un evento debería tener información suficiente para que consumidores puedan procesarlo.

Ejemplo:

```json
{
  "eventId": "123",
  "eventType": "PaymentCompleted",
  "timestamp": "...",
  "version": 1,
  "paymentId": "P100",
  "orderId": "O100",
  "amount": 150.00
}
```

9.2 Schema Evolution

Los eventos evolucionan.

Debemos considerar:

- backward compatibility;
- forward compatibility;
- optional fields;
- schema versioning.

Herramientas:

- Avro.
- Protobuf.
- JSON Schema.
- Schema Registry.

---

10. KAFKA

Esta debe ser una de las áreas de mayor profundidad.

10.1 ¿Qué es Kafka?

Kafka es una plataforma distribuida de event streaming basada en un log append-only.

Conceptualmente:

```
None
Topic
 |
 +-- Partition 0
 |     0 1 2 3 4 5
 |
 +-- Partition 1
 |     0 1 2 3 4
 |
 +-- Partition 2
```

```
0 1 2 3
```

10.2 Broker

Un broker es un servidor Kafka que almacena y sirve datos.

Un cluster tiene múltiples brokers.

10.3 Topic

Un topic es una categoría lógica de eventos.

Ejemplo:

```
None
payment-events
order-events
customer-events
```

10.4 Partition

Este concepto es crítico.

Una partition es una secuencia ordenada de records.

La partition es también una unidad fundamental de:

parallelism in Kafka consumer groups.

10.5 Consumer Groups

Supongamos:

```
None
Topic
4 partitions

Consumer Group
C1
C2
C3
C4
```

Puede existir aproximadamente:

```
None
P0 -> C1
P1 -> C2
P2 -> C3
P3 -> C4
```

Si tenemos:

```
None
4 partitions
8 consumers
```

solo 4 consumers pueden tener partitions asignadas en ese consumer group; los otros 4 estarán idle.

Regla conceptual:

> Maximum active consumers per consumer group for a topic ≈ number of partitions.

Esta era una de las preguntas críticas de la entrevista.

10.6 Partition Key

Kafka puede utilizar una key para determinar la partition.

Ejemplo:

```
None
key = customerId
```

Esto permite mantener eventos del mismo customer en la misma partition.

Importante:

> Kafka ordering is guaranteed within a partition, not globally across a topic.

10.7 Offset

Cada record tiene un offset dentro de su partition.

```
None
Partition 0

Offset
0
1
2
3
4
```

El consumer mantiene una posición.

Esto permite:

- resume;
- replay;
- recovery;
- consumer lag monitoring.

10.8 Replication

Las partitions pueden replicarse entre brokers.

```
None
Partition 0

Broker 1 -> Leader
Broker 2 -> Replica
Broker 3 -> Replica
```

Esto proporciona fault tolerance.

10.9 Consumer Lag

Lag representa aproximadamente cuánto procesamiento está atrasado un consumer respecto al final disponible del log.

Si:

```
None
Latest offset = 1000
Consumer offset = 700
```

Lag ≈ 300 records.

High lag puede indicar:

- insufficient consumers;
- slow processing;
- downstream bottleneck;
- insufficient partitions;
- resource constraints.

10.10 At-Most-Once

Puede perder mensajes.

```
None
Read
Commit offset
```

```
Process
```

Si falla después del commit:

```
None
Message may not be processed
```

10.11 At-Least-Once

Se prioriza no perder el mensaje, pero pueden existir duplicates.

```
None
Process
Crash before commit
Restart
Process again
```

Por eso necesitamos:

> Idempotent consumers.

10.12 Exactly-Once

Exactly-once es más complejo que simplemente "el mensaje se procesa una vez".

Hay que distinguir:

> Exactly-once processing

de:

> Exactly-once end-to-end business side effects.

Kafka soporta transacciones y mecanismos de exactly-once semantics en escenarios específicos, pero una arquitectura completa debe considerar también sistemas externos.

Ejemplo:

```
None
Kafka
  |
  v
Processor
  |
  v
External Database
```

No debemos asumir automáticamente que Kafka puede garantizar exactamente una escritura en una DB externa sin un diseño adicional.

10.13 Kafka Design Questions

Para cada topic debemos pensar:

- ¿Qué representa?
- ¿Quién produce?
- ¿Quién consume?
- ¿Cuántas partitions?
- ¿Cuál partition key?
- ¿Necesitamos ordering?
- ¿Cuál retention?
- ¿Necesitamos replay?
- ¿Necesitamos compaction?
- ¿Qué schema?
- ¿Qué compatibility policy?
- ¿Qué happens on poison messages?
- ¿Cómo monitoreamos lag?

---

11. KAFKA VS RABBITMQ

Kafka

Optimizado para:

- high throughput;
- event streaming;

- replay;
- persistent logs;
- multiple consumers;
- partition-based parallelism.

RabbitMQ

Optimizado para:

- message routing;
- queues;
- acknowledgments;
- work distribution;
- routing patterns.

RabbitMQ utiliza conceptos como:

- exchanges;
- queues;
- bindings;
- routing keys;
- acknowledgments;
- dead-letter exchanges.

Pregunta correcta:

> "Which one is better?"

No existe una respuesta universal.

La pregunta correcta es:

> "Which communication model does the system require?"

---

12. BACKPRESSURE

Backpressure aparece cuando el productor genera datos más rápido de lo que el consumidor puede procesarlos.

```
None
Producer
  |
  | 10,000 msg/sec
```

```
  v
Kafka
  |
  | 2,000 msg/sec
  v
Consumer
```

Se acumula backlog.

Soluciones:

- Increase consumer parallelism.
- Increase partitions.
- Batch processing.
- Optimize consumer.
- Autoscaling.
- Rate limiting.
- Buffering.
- Load shedding.
- Backpressure-aware processing.

Kafka ayuda a absorber temporalmente bursts porque los eventos permanecen almacenados.

---

13. DATA ARCHITECTURE

13.1 OLTP

Optimizado para operaciones transaccionales.

Ejemplo:

```
None
Create payment
Update customer
Insert order
```

Características:

- frequent writes;

- transactions;
- normalized models;
- low latency.

13.2 OLAP

Optimizado para analytics.

Ejemplo:

```
None
Revenue by country
Sales by month
Customer trends
```

13.3 Data Lake

Almacena grandes cantidades de datos, frecuentemente sobre object storage.

Puede almacenar:

- structured;
- semi-structured;
- unstructured.

Ejemplo:

```
None
S3
ADLS
GCS
```

13.4 Data Warehouse

Diseñado para análisis estructurado y SQL.

Ejemplos:

- Snowflake.
- BigQuery.
- Redshift.

13.5 Data Mart

Subset orientado a un área de negocio.

Ejemplo:

```
None
Enterprise Warehouse
       |
       +-- Finance Mart
       +-- Marketing Mart
       +-- Sales Mart
```

13.6 Data Swamp

Un Data Lake sin:

- governance;
- metadata;
- ownership;
- quality;
- documentation;
- lifecycle management.

Resultado:

> Data exists but cannot be trusted or found.

13.7 Data Lakehouse

Combina:

```
None
Data Lake
+
Warehouse capabilities
```

Busca proporcionar:

- ACID transactions;
- schema enforcement;
- SQL analytics;
- scalable object storage;
- multiple processing engines.

Tecnologías relevantes:

- Delta Lake.
- Apache Iceberg.
- Apache Hudi.

---

14. MEDALLION ARCHITECTURE

Bronze

Raw data.

```
None
Kafka
 |
 v
Bronze
```

Características:

- minimally transformed;
- replayable;
- immutable whenever practical;
- source-oriented.

Silver

Cleaned and standardized.

```
None
Bronze
   |
   v
Silver
```

Operaciones:

- deduplication;
- validation;
- normalization;
- schema enforcement;
- enrichment.

Gold

Business-ready.

```
None
Silver
   |
   v
Gold
```

Ejemplo:

```
None
daily_revenue
customer_metrics
payment_summary
```

14.1 Why Medallion?

Permite separar:

```
None
Raw
  |
Clean
  |
Business
```

Esto facilita:

- debugging;
- lineage;
- reprocessing;
- data quality;
- auditing.

---

15. CDC — CHANGE DATA CAPTURE

CDC captura cambios realizados en una base de datos.

Tipos:

```
None
INSERT
UPDATE
DELETE
```

En sistemas modernos, CDC puede leer logs de la base de datos.

Ejemplo:

```
None
Operational DB
      |
      | CDC
      v
   Kafka
      |
```

```
      v
Data Platform
```

15.1 Polling vs CDC

Polling

```
None
SELECT * FROM table
WHERE updated_at > last_timestamp
```

Problemas:

- query overhead;
- latency;
- deleted records pueden ser difíciles;
- cambios rápidos pueden complicarse;
- mayor carga sobre DB.

CDC

```
None
DB Transaction Log
        |
        v
       CDC
        |
        v
      Kafka
```

Ventajas:

- near real-time;
- menor query overhead;
- captura deletes;
- mantiene orden relativo del log según tecnología/configuración;
- excelente para replication pipelines.

Herramientas:

- Debezium.
- AWS DMS.
- Datastream.
- Azure Data Factory / ecosystem tools.

---

16. BATCH VS STREAMING

Batch

Procesa datos acumulados.

```
None
Data
 |
 v
Scheduled Job
 |
 v
Result
```

Ejemplo:

> Process yesterday's transactions at 2 AM.

Streaming

Procesa eventos continuamente.

```
None
Event
Event
Event
Event
 |
 v
Stream Processor
```

```
 |
 v
Result
```

Ejemplo:

> Detect fraudulent payment within seconds.

16.1 Batch Processing Concepts

- Jobs.
- Scheduling.
- Dependencies.
- Retries.
- Backfills.
- Reprocessing.
- Partitioning.
- Checkpoints.

16.2 Streaming Concepts

- Event time.
- Processing time.
- Watermarks.
- Windows.
- State.
- Late events.
- Checkpointing.
- Exactly-once processing.

---

17. STREAM WINDOWS

Tumbling Window

Ventanas sin overlap.

```
None
00:00 - 01:00
01:00 - 02:00
02:00 - 03:00
```

Sliding / Hopping Window

Las ventanas se superponen.

```
None
00:00 - 01:00
00:15 - 01:15
00:30 - 01:30
```

Session Window

Agrupa eventos relacionados con una sesión de actividad.

Ejemplo:

```
None
User active
 |
Event
 |
Event
 |
Event
 |
30 min inactivity
 |
Session ends
```

---

18. EVENT TIME VS PROCESSING TIME

Event Time

Momento en que ocurrió el evento.

Processing Time

Momento en que el sistema procesó el evento.

Pueden ser diferentes.

```
None
Event occurred: 10:00
Network delay
Processed: 10:05
```

Esto es fundamental para streaming.

---

19. WATERMARKS

Un watermark permite al sistema estimar hasta qué punto puede considerar completos los eventos de un stream.

Sirve para manejar:

> Late-arriving events.

---

20. SPARK

Apache Spark es un framework para distributed data processing.

Conceptos fundamentales:

- Driver.
- Executors.
- Cluster manager.
- Jobs.
- Stages.
- Tasks.
- Partitions.
- Shuffle.

20.1 Spark Driver

Coordina el procesamiento.

20.2 Executors

Ejecutan las tareas.

```
None
              Driver
                 |
        +--------+--------+
        |        |        |
    Executor Executor Executor
```

20.3 Spark Partitions

Los datos se dividen en partitions para procesamiento paralelo.

Importante:

> Partitioning exists at multiple layers.

Kafka partitions ≠ Spark partitions.

Pero pueden influenciarse mutuamente en un pipeline.

20.4 Shuffle

Ocurre cuando Spark necesita redistribuir datos entre executors.

Ejemplo:

```
None
GROUP BY customer_id
JOIN
ORDER BY
```

Shuffle puede ser costoso por:

- network I/O;
- disk I/O;
- serialization;
- skew.

20.5 Data Skew

Cuando algunos partitions contienen mucho más trabajo que otros.

```
None
Partition 1 -> 1 GB
Partition 2 -> 1 GB
Partition 3 -> 1 GB
Partition 4 -> 100 GB
```

El partition 4 se convierte en bottleneck.

---

21. FILE FORMATS

CSV

Simple, pero:

- larger size;
- weak schema;
- slow parsing.

JSON

Flexible y legible.

Pero puede ser pesado para analytics a escala.

Avro

- Binary.
- Schema-based.
- Bueno para eventos y messaging.

Parquet

Columnar format.

Excelente para analytics.

```
None
Columnar
Name | Name | Name
Age  | Age  | Age
```

Permite leer solamente columnas necesarias.

---

22. DATABASE ARCHITECTURE

No existe una "mejor base de datos".

La selección depende del access pattern.

22.1 Relational Databases

Ejemplos:

- PostgreSQL.
- MySQL.
- SQL Server.
- Oracle.

Características:

- structured schema;

- SQL;
- ACID;
- joins;
- transactions.

Usar cuando:

- consistency matters;
- relationships are important;
- transactional workloads.

22.2 Document Databases

Ejemplo:

- MongoDB.

Datos almacenados como documentos.

Útil cuando:

- schema flexibility;
- aggregate-oriented access;
- document-centric applications.

22.3 Key-Value

Ejemplos:

- Redis.
- DynamoDB, en determinados modelos.

Excelente para:

```
None
key -> value
```

Casos:

- caching;
- sessions;

- high-speed lookups.

22.4 Wide Column

Ejemplos:

- Cassandra.
- ScyllaDB.

Diseñadas para:

- high write throughput;
- horizontal scaling;
- distributed workloads.

22.5 Search Engines

Ejemplos:

- Elasticsearch.
- OpenSearch.

Optimized for:

- text search;
- filtering;
- aggregations;
- log analytics.

No deben asumirse como reemplazo universal de una transactional database.

22.6 Vector Databases

Diseñadas para almacenar y consultar embeddings/vector representations.

Casos:

- semantic search;
- RAG;

- recommendation;
- similarity search.

Conceptos:

- embeddings;
- dimensions;
- similarity;
- cosine similarity;
- approximate nearest neighbor.

---

23. CACHING

Cache-Aside

```
None
Application
    |
    v
Cache?
 /    \
Hit   Miss
 |      |
 v      v
Return  DB
         |
         v
       Cache
```

Ventajas:

- reduce database load;
- low latency.

Problemas:

- stale data;
- cache invalidation;
- cache stampede.

---

24. DATA MODELING

Normalization

Reduce duplication y dependencia.

Denormalization

Duplica información deliberadamente para mejorar lectura.

24.1 Star Schema

```
None
              Date
               |
Customer -- Fact Sales -- Product
               |
            Location
```

Fact table:

- measures;
- foreign keys.

Dimension tables:

- descriptive attributes.

24.2 Slowly Changing Dimensions

Permiten manejar cambios históricos en dimensiones.

Ejemplo:

Customer cambia de ciudad.

Necesitamos decidir:

¿Sobrescribimos o preservamos el historial?

---

25. DATA GOVERNANCE

Incluye:

- Data ownership.
- Data catalog.
- Data lineage.
- Data quality.
- Data classification.
- Data retention.
- Access control.
- Auditability.

25.1 Data Lineage

Permite conocer:

```
None
Source
  |
  v
CDC
  |
  v
Kafka
  |
  v
Bronze
  |
  v
Silver
  |
  v
Gold
  |
```

```
  v
BI
```

Pregunta:

> Where did this number come from?

Lineage debe permitir responderla.

25.2 Data Quality

Dimensiones comunes:

- Accuracy.
- Completeness.
- Consistency.
- Uniqueness.
- Timeliness.
- Validity.

---

26. ORCHESTRATION

Un orchestrator controla workflows.

Ejemplo:

```
None
Job A
 |
 v
Job B
 |
 +----> Job C
 |
 v
Job D
```

Conceptos:

- DAG.
- Dependencies.
- Retry.
- Scheduling.
- Failure handling.
- Backfill.
- Monitoring.

Herramientas:

- Airflow.
- Dagster.
- Databricks Workflows.
- AWS Step Functions.

---

27. CLOUD & INFRASTRUCTURE

Containers

Docker empaqueta:

- application;
- runtime;
- dependencies.

Kubernetes

Kubernetes administra workloads containerizados.

Conceptos:

- Pod.
- Deployment.
- Service.
- ConfigMap.
- Secret.
- Ingress.
- Horizontal Pod Autoscaler.
- Namespace.

Serverless

Permite ejecutar código sin administrar servidores directamente.

Trade-offs:

- cold starts;
- execution limits;
- vendor coupling;
- event-driven design;
- pay-per-use.

Infrastructure as Code

Ejemplos:

- Terraform.
- CloudFormation.
- OpenTofu.

Principios:

- reproducibility;
- version control;
- automation;
- reviewable infrastructure changes.

---

28. OBSERVABILITY

Logs

Qué ocurrió.

Metrics

Cuánto / con qué frecuencia.

Traces

Dónde pasó una operación distribuida.

28.1 Distributed Tracing

Ejemplo:

```
None
Request
 |
API Gateway
 |
Order Service
 |
Payment Service
 |
Database
```

Cada request debería poder correlacionarse.

Conceptos:

- Trace ID.
- Span ID.
- Correlation ID.

---

29. SECURITY

Conceptos fundamentales:

- Authentication.
- Authorization.
- IAM.
- Least privilege.
- Encryption at rest.
- Encryption in transit.
- Secrets management.
- OAuth2.

- OIDC.
- mTLS.

---

30. TESTING

Unit Testing

Test de componentes aislados.

Integration Testing

Test de interacción entre componentes.

Contract Testing

Valida que productor y consumidor respeten el contrato.

Especialmente importante en microservices.

Load Testing

Evalúa comportamiento bajo carga.

Performance Testing

Evalúa latency y throughput.

Chaos Testing

Introduce fallos deliberadamente para validar resilience.

---

31. ARCHITECTURE DIAGRAMMING

Durante una entrevista de arquitectura, no comenzar dibujando tecnologías.

Primero identificar:

1. Users.

2. Entry point.
3. Core business services.
4. Data stores.
5. Communication.
6. Async processing.
7. External dependencies.
8. Observability.
9. Security.
10. Failure handling.

---

32. PAYMENT ARCHITECTURE — INTERVIEW EXERCISE

Caso:

> Design a payment processing platform.

Primero identificar requisitos.

Functional

- Receive payment.
- Validate payment.
- Persist transaction.
- Process payment.
- Return result.
- Publish event.
- Support downstream consumers.

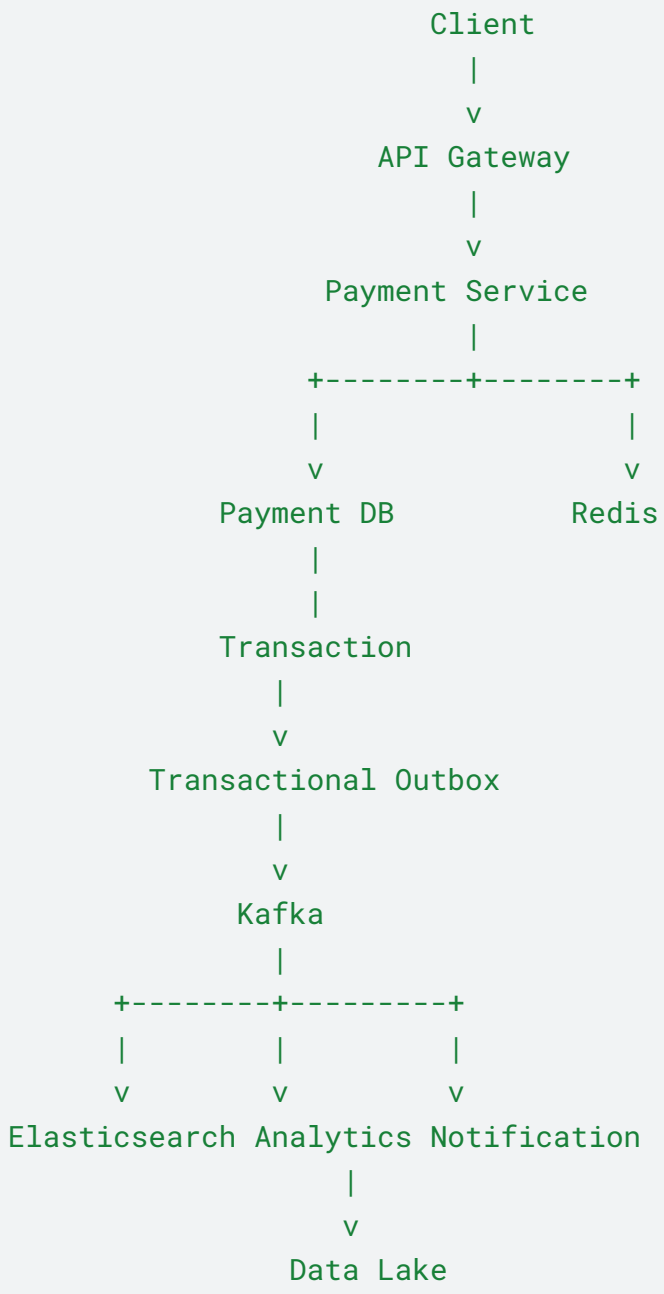
Non-functional

Preguntar:

- TPS?
- Latency?
- Availability?
- Consistency?
- Recovery?
- Audit requirements?
- Retention?
- PCI requirements?
- Geographic distribution?

32.1 Possible Architecture

None



El objetivo de la entrevista no es dibujar exactamente esta arquitectura.

El objetivo es poder explicar:

> Why is each component there?

---

33. PAYMENT FAILURE SCENARIOS

Database failure

¿Qué ocurre?

- Retry?
- Fail fast?
- Return error?
- Queue request?
- Failover?

Kafka failure

Si DB commit succeeds pero Kafka publish fails:

> Transactional Outbox.

Duplicate payment request

> Idempotency key.

Payment provider unavailable

> Timeout + Circuit Breaker + Retry policy + Fallback según el caso.

Slow downstream service

> Timeout + Bulkhead + Circuit Breaker.

---

34. DATA PLATFORM — INTERVIEW EXERCISE

Caso:

> Design a real-time analytics platform.

```
None
Operational DB
      |
      | CDC
      v
    Kafka <----- Application Events
      |
      v
   Bronze
      |
      v
   Silver
      |
      v
    Gold
      |
      +----> Warehouse / Serving DB
      |
      +----> BI API
```

34.1 Bronze

Debe permitir:

- replay;
- audit;
- raw history;
- reprocessing.

34.2 Silver

Debe garantizar:

- schema validation;

- deduplication;
- data quality;
- normalization.

34.3 Gold

Debe representar:

- business metrics;
- aggregates;
- consumption models.

---

35. HOW TO ANSWER AN ARCHITECTURE INTERVIEW

Utilizar siempre esta secuencia:

Step 1 — Clarify Requirements

Preguntar:

- What is the expected traffic?
- What is the latency requirement?
- What is the availability requirement?
- Is consistency critical?
- Is the system real-time?
- How much data?
- What are retention requirements?

Step 2 — Define High-Level Architecture

Primero:

```
None
Client
  |
```

```
Gateway
 |
Services
 |
Data
```

No empezar con detalles.

Step 3 — Identify Communication Model

Preguntar:

> Should this communication be synchronous or asynchronous?

Synchronous:

- REST.
- gRPC.

Asynchronous:

- Kafka.
- RabbitMQ.
- SQS.
- Pub/Sub.

Step 4 — Define Data Ownership

Preguntar:

> Which service owns this data?

Evitar shared database ownership entre microservices.

Step 5 — Define Consistency

Preguntar:

> Do we require strong consistency or is eventual consistency acceptable?

Step 6 — Define Failure Handling

Para cada dependencia:

```
None
Timeout
Retry
Circuit Breaker
Fallback
Bulkhead
```

No aplicar todos automáticamente.

Cada patrón debe tener una razón.

Step 7 — Define Scalability

Preguntar:

- What scales?
- Horizontally?
- Where is the bottleneck?
- What is the partition key?
- Can consumers scale?
- Where is the state?

Step 8 — Define Observability

Agregar:

```
None
Logs
Metrics
Tracing
Alerts
```

Step 9 — Define Security

Considerar:

- authentication;
- authorization;
- encryption;
- secrets;
- network boundaries;
- audit.

Step 10 — Explain Trade-offs

Nunca terminar solamente con:

> "This is the architecture."

Terminar explicando:

> "I chose X because..."
>
> "The trade-off is..."
>
> "If the requirement changes to Y, I would reconsider..."

---

36. HIGH-PRIORITY INTERVIEW TOPICS

Por el tipo de entrevista descrita, estos conceptos deberían tener prioridad:

Priority 1 — Must Know

Distributed Systems

- CAP.
- Consistency.
- Availability.
- Partial failures.
- Timeouts.
- Retries.
- Idempotency.

Kafka

- Broker.
- Topic.
- Partition.
- Offset.
- Consumer Group.
- Partition key.
- Consumer parallelism.
- Ordering.
- Replication.
- Consumer lag.
- Delivery semantics.
- Exactly-once concepts.

Microservices

- Service boundaries.
- Database per service.
- Service discovery.
- API Gateway.
- gRPC.
- REST.

Distributed Transactions

- Saga.
- Orchestration.
- Choreography.
- Compensation.
- Outbox.
- Inbox.

Data Engineering

- CDC.
- Batch.
- Streaming.
- Bronze.
- Silver.
- Gold.
- Data Lake.
- Warehouse.
- Lakehouse.

---

37. SECOND PRIORITY

- Spark architecture.
- Shuffle.
- Partitioning.
- Data skew.
- Streaming windows.
- Watermarks.
- Event time.
- Parquet.
- Avro.
- Schema Registry.
- Redis.
- Elasticsearch.
- Database selection.
- Caching.
- CQRS.
- Event Sourcing.

---

38. THIRD PRIORITY

- Kubernetes.
- Terraform.
- Service Mesh.
- OpenTelemetry.
- Chaos Engineering.
- Data Mesh.
- Data Fabric.
- Iceberg.
- Hudi.
- Advanced cloud services.

---

39. THE FIVE QUESTIONS FOR EVERY ARCHITECTURE

Para cualquier componente que aparezca en un diagrama, practicar estas cinco preguntas:

1. Why is it there?

¿Por qué existe?

2. What problem does it solve?

¿Qué problema resuelve?

3. How does it scale?

¿Cómo escala?

4. What happens when it fails?

¿Qué ocurre cuando falla?

5. What are the trade-offs?

¿Qué sacrificamos al utilizarlo?

---

40. STUDY METHOD

No estudiar solamente leyendo.

Para cada concepto utilizar:

```
None
Understand
    ↓
Explain
    ↓
Draw
    ↓
Compare
    ↓
Apply
    ↓
Defend
```

Ejemplo Kafka:

Understand

¿Qué es una partition?

Explain

Explicarlo en 60 segundos.

Draw

Dibujar:

```
None
Topic
 |
+-- P0
+-- P1
+-- P2
```

Compare

Kafka partition vs Spark partition.

Apply

Diseñar un sistema con 100 consumers.

Defend

¿Por qué elegirías 20 partitions?

---

41. INTERVIEW FLASH QUESTIONS

Practicar estas preguntas sin consultar documentación.

Architecture

- What is the difference between software, solution and system architecture?
- Monolith vs modular monolith?
- Microservices trade-offs?
- When would you not use microservices?
- REST vs gRPC?
- Synchronous vs asynchronous communication?

Distributed Systems

- What is CAP?
- What is eventual consistency?
- What is a partial failure?

- Why are retries dangerous?
- What is idempotency?

Kafka

- What is a partition?
- Why does partition count matter?
- How does consumer parallelism work?
- What is a consumer group?
- What is an offset?
- What is consumer lag?
- How does Kafka guarantee ordering?
- What happens if a consumer crashes?
- At-most-once vs at-least-once?
- What does exactly-once mean?
- How would you design Kafka topics?
- How would you choose a partition key?
- Kafka vs RabbitMQ?

Microservices

- What is Saga?
- Orchestration vs choreography?
- What is a compensating transaction?
- Why use Outbox?
- What is Inbox?
- What is Service Discovery?
- Why database-per-service?

Data

- What is CDC?
- CDC vs polling?
- Data Lake vs Warehouse?
- Data Lake vs Lakehouse?
- Bronze vs Silver vs Gold?
- Batch vs streaming?
- Event time vs processing time?
- What are streaming windows?
- What is a watermark?

Spark

- Driver vs Executor?
- What is a partition?

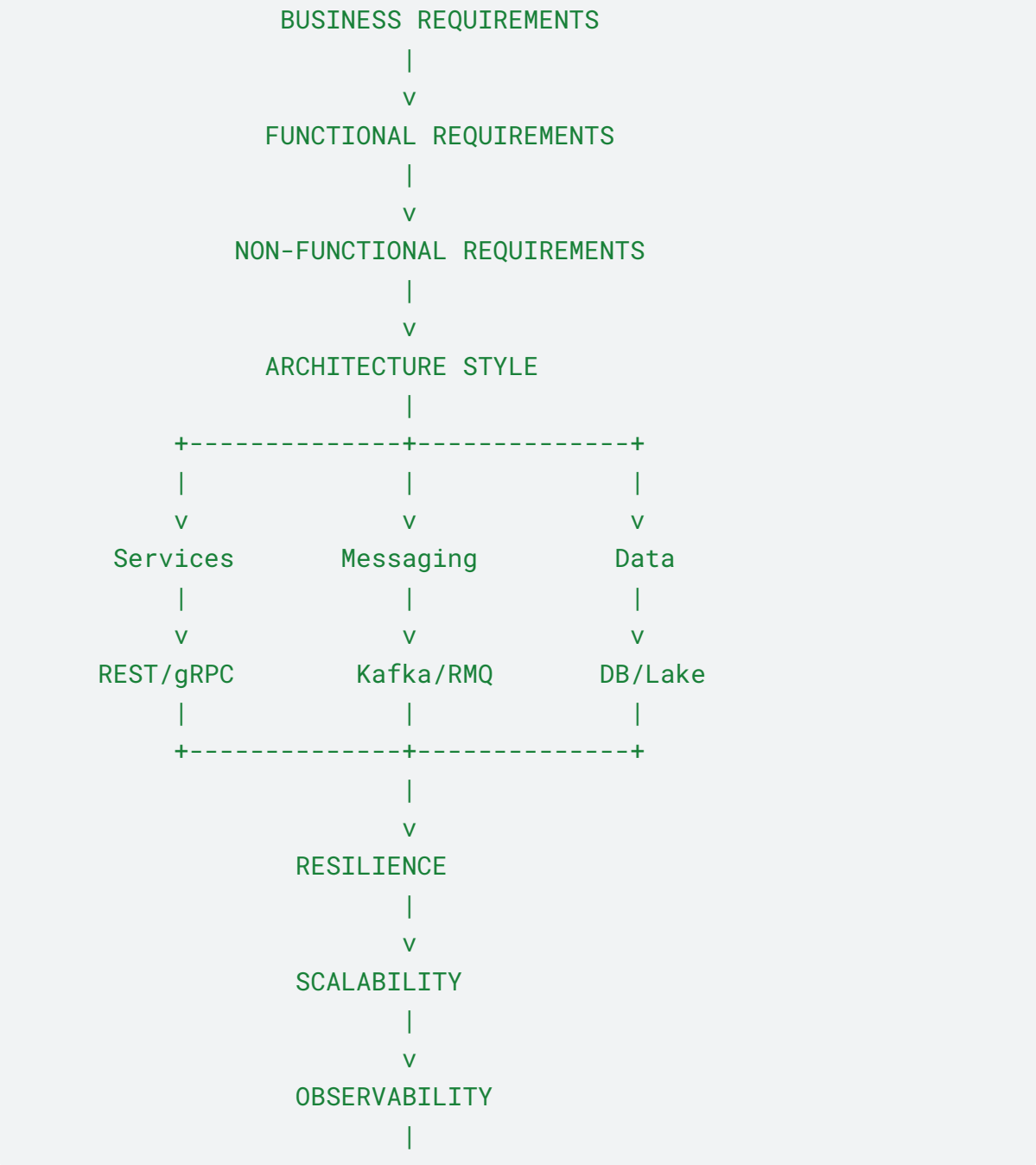
- What is shuffle?
- What is data skew?
- How would you optimize a Spark job?

---

42. FINAL MENTAL MODEL

Cuando recibas un architecture problem, pensar en este orden:

None



```
        v
   SECURITY
        |
        v
  TRADE-OFFS
```

La arquitectura final no es simplemente:

> "Use Kafka + Spark + Databricks + PostgreSQL."

La respuesta Senior es:

> "Given the throughput, latency, consistency and reliability requirements, I would separate the transactional and analytical workloads. The transactional path would remain synchronous and ACID. Once the transaction commits, I would propagate changes asynchronously using an event-driven architecture. To avoid the dual-write problem, I would use a transactional outbox or CDC depending on the source of truth. Kafka would provide the durable ingestion buffer and consumer parallelism through partitions. Downstream processing would use streaming or batch depending on the latency requirement. The data platform would use Bronze/Silver/Gold layers to preserve raw data, enforce quality, and expose business-ready datasets."

Ese cambio —de **nombrar tecnologías** a **explicar decisiones arquitectónicas**— es uno de los objetivos principales de este plan.

---

43. NEXT LEVEL — SYSTEM DESIGN PRACTICE

Una vez dominados los fundamentos, practicar estos casos:

Case 1

Design a payment processing system.

Case 2

Design a real-time fraud detection platform.

Case 3

Design an order management system.

Case 4

Design a CDC-based data platform.

Case 5

Design a real-time analytics platform.

Case 6

Design a notification system.

Case 7

Design a log ingestion platform.

Case 8

Design a recommendation system.

Case 9

Design a distributed file processing platform.

Case 10

Design a high-throughput event-driven microservices platform.

Para cada ejercicio utilizar el mismo framework:

```
None
Requirements
     ↓
Scale
     ↓
Architecture
     ↓
Data
     ↓
Communication
     ↓
Consistency
     ↓
```

```
Failure Handling
     ↓
Scalability
     ↓
Observability
     ↓
Security
     ↓
Trade-offs
```

End